

Project Report For CS661

CryptoLens

An Interactive Visual Analytics System for
Cryptocurrency Market Intelligence

Group Number – 13

Group Members

Name	Roll No.
Soumya Jhunhunwala	241034
Vivek Singh	241186
Arpit Kumar Singh	240194
Aman	240097
Pratyush Sandhwar	240788
Lakshya Arukiya	240589
Aadi Sehal	240005
Anandan Iyer	240117
Tulika	241106

Contents

Introduction	4
Basic Terms Used in This Report	4
The Five Views of CryptoLens	5
1 Executive Summary	6
2 Introduction to the Problem	6
2.1 Background and Motivation	6
2.2 Problem Statement	6
2.3 Scope and Objectives	7
2.4 Theoretical Basis: Measuring Risk and Co-Movement	8
2.5 Analyzing Crypto Market Data	8
3 Tasks	9
3.1 Tasks Addressed by the Visualization Tool	9
4 Proposed Methodology of Dedicated Dashboard Views and Results	9
4.1 Purpose of Visualization in Market Analysis	10
4.2 Dedicated Dashboard Views and Their Functionalities	10
4.3 Tools and Technologies	11
4.4 Outcome	11
5 Data Sources and Preprocessing	11
5.1 Overview of Datasets	11
5.1.1 Binance Vision: Historical 1-Minute Klines	11
5.1.2 Binance Live Depth Stream	12
5.2 Data Cleaning and Aggregation	12
5.2.1 Heterogeneous Header Detection	12
5.2.2 Timestamp Unit Normalisation	12
5.2.3 Deduplication on Insert	12
5.2.4 On-the-Fly Multi-Resolution Aggregation	13
5.3 Feature Engineering	13
5.3.1 Log Returns	13
5.3.2 Rolling Volatility	13
5.3.3 Distribution Moments	13
5.3.4 Scenario Sampling	13
5.4 Specific Dataset Notes	14
5.4.1 Missing Months and Late Listings	14
5.4.2 Response Size Guards	14
5.4.3 Synthetic vs. Real Depth	14
6 Application Architecture	14
6.1 Framework: A Three-Tier Web Application	14
6.2 Core Components: Layout, State, and Callbacks	15
6.3 UI and Styling Elements	15
7 Overall Application Story: Understanding Crypto Market Behavior	15

8	View-Specific Analysis and Features	16
8.1	The Time Machine (Temporal Explorer)	16
8.1.1	Interaction and Controls	16
8.1.2	Semantic Zoom (Auto Resolution)	16
8.1.3	Candlestick and Volume Visualization	17
8.1.4	Backend and Plot Logic	17
8.1.5	Insights and Use Cases	17
8.2	Market Depth Visualiser	18
8.2.1	Functionality Overview	18
8.2.2	Live Streaming and Resilience	18
8.2.3	Historical Reconstruction	18
8.2.4	Interactive Elements	19
8.3	The What-If Simulator	19
8.3.1	Functionality Overview	19
8.3.2	Core Visualization: The Fan Chart	19
8.3.3	Scan Algorithm and Statistics	20
8.3.4	Interactive Elements and Linking	20
8.4	Volatility Engine	20
8.4.1	Functionality Overview	20
8.4.2	Scenario Modelling	21
8.4.3	Core Visualizations	21
8.4.4	Insights and Use Cases	22
8.5	Cross-Asset Correlation Matrix	22
8.5.1	Core Visualizations	22
8.5.2	Interactive Elements and Drill-Down	22
8.5.3	Backend Logic	23
8.5.4	Insights and Use Cases	23
8.6	Member Contributions	23
9	Conclusion	24
10	Code and Deployment Links	24

Introduction

Cryptocurrency markets never close. Prices change every second, buyers and sellers come and go all the time, and the way different coins move together can change suddenly during a market crash. Inside all this activity there are useful patterns but the raw data is simply too large and too fast for a person to read directly. Five years of prices for five coins means more than ten million rows of numbers.

This project presents **CryptoLens**, an interactive visual analytics tool for exploring five years of minute-by-minute market data for five major crypto assets: **Bitcoin (BTC)**, **Ethereum (ETH)**, **Binance Coin (BNB)**, **Solana (SOL)**, and **XRP**. The goal is to let a user move easily between the big picture (years of history) and the fine detail (a single hour), measure how risky an asset is, test trading ideas against the full history, and see how the five assets move together all without writing any code.

The tool is built as a three-part web application. A Python **FastAPI** backend serves data and statistics from an embedded **DuckDB** database; a small **Node.js** relay server connects the browser to Binance’s live order-book feed; and a **React** + **Plotly/D3** frontend draws every chart. The historical data comes from the free, public **Binance Vision** data portal, and the live order book comes from Binance’s public WebSocket stream.

Basic Terms Used in This Report

Before describing the system, we briefly explain the market terms that appear throughout this report. Readers already familiar with trading charts can skip this part. (Deeper statistical terms such as Value-at-Risk and kurtosis are explained later, in the sections where they are actually used.)

Candles (OHLCV). A price chart does not show every single trade. Instead, time is cut into fixed intervals (one minute, one hour, one day, ...) and each interval is summarised by one *candle* with five numbers, together called **OHLCV**: the **Open** (first price in the interval), **High** (highest price), **Low** (lowest price), **Close** (last price), and **Volume** (how much was traded). The thick “body” of the candle runs from open to close, and the thin lines (“wicks”) reach up to the high and down to the low. A green candle means the price ended higher than it started; a red candle means it ended lower. Figure 1 shows both cases.

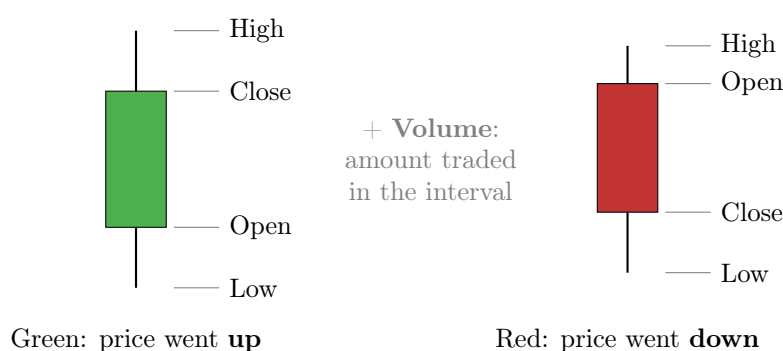


Figure 1: Anatomy of a candlestick. Each candle summarises one time interval with its Open, High, Low, and Close prices; Volume records how much was traded.

Resolution (timeframe). The length of the interval each candle covers. The same history can be viewed as monthly candles (good for a five-year overview) or minute candles (good for studying a single volatile hour). In this report we write these as **1m** (one minute), **1h**, **1d**, **1w**, and **1M** (one month).

Order book, bids and asks. At every moment, an exchange keeps a live list of all waiting orders, called the **order book**. Orders from buyers are called **bids** and sit just below the current price; orders from sellers are called **asks** and sit just above it. The gap between the best bid and the best ask is the *spread*, and the point halfway between them is the *mid price*.

Market depth, support walls and resistance walls. If we add up the order sizes level by level as we move away from the current price, we get the **depth chart** the shape of standing buy and sell interest. A large pile of buy orders at one price acts like a floor that is hard for the price to fall through: a **support wall**. A large pile of sell orders acts like a ceiling that is hard to rise through: a **resistance wall**. In a depth chart these walls show up as sudden vertical jumps (Figure 2).

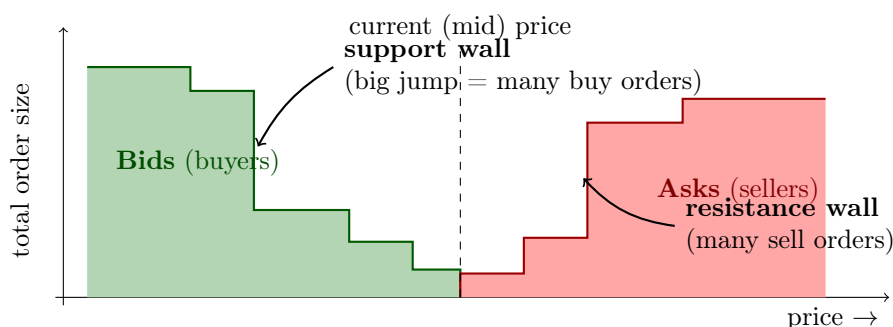


Figure 2: A depth chart. Buy orders (bids) pile up on the left of the current price and sell orders (asks) on the right. Sudden jumps in the curve are “walls” support on the buy side, resistance on the sell side.

Returns. A *return* is simply the percentage change of the price over one interval. Almost all risk and correlation analysis works on returns instead of raw prices, because returns from different assets and different times can be compared directly.

Volatility. A simple word for “how much the price jumps around.” An asset whose daily moves are usually small is low-volatility; one that regularly moves several percent in a day is high-volatility. Volatility is the standard measure of risk.

Correlation. A number between -1 and $+1$ that says how much two assets move together. $+1$ means they always move in the same direction, 0 means their moves are unrelated, and -1 means they always move in opposite directions.

Brushing and linked views. An interaction technique from visual analytics: the user drags (“brushes”) to select a time window in one chart, and every other chart on the screen instantly updates to show the same window. This keeps all views “linked” and telling one consistent story.

The Five Views of CryptoLens

CryptoLens is organised into **five linked views**, each looking at the data from a different angle:

- **The Time Machine:** a temporal explorer. Brush across five years of history and zoom down to minute-level candles.
- **Market Depth Visualiser:** the order book live from Binance, or rebuilt for a historical moment.
- **The What-If Simulator:** a pattern tester. Ask “what usually happens after a move like this?” and see the answer from the full history.
- **Volatility Engine:** risk analytics under normal, stress, and crash market conditions.

- **Cross-Asset Correlation Matrix:** how the five assets move together, shown as a heatmap and as a network.

Every view is driven by one shared, brushed time window: selecting a period in one panel instantly refocuses all the others. This “brush-and-link” design turns the dashboard into a single connected workspace instead of a set of separate charts. CryptoLens is aimed at traders, researchers, and students who want to turn raw high-frequency market data into understanding.

1 Executive Summary

CryptoLens is an interactive visual analytics system for cryptocurrency market data covering 2020 to 2024 at one-minute resolution for five major assets (BTC, ETH, BNB, SOL, XRP). It is built with Python (FastAPI, pandas, NumPy, SciPy), DuckDB, Node.js, and a React + Plotly/D3 frontend. The tool lets users explore price history, measure tail risk, back-test price patterns, inspect the order book, and study how assets move together all through a set of linked, brushable views.

The system loads roughly **10.5 million** one-minute candles from the public Binance Vision data portal into an embedded columnar database, and combines them on the fly into any of five resolutions (minute, hourly, daily, weekly, monthly) using DuckDB’s `time_bucket` function. One shared time window links all five views, so a single brush gesture drives the whole dashboard. This report describes the project’s design, its data pipeline, its statistics engine, and the key features of each view.

2 Introduction to the Problem

2.1 Background and Motivation

Crypto markets trade 24 hours a day, 365 days a year there is no closing bell. This produces very long, unbroken time series: a single asset generates 525,600 one-minute candles per year, so a five-asset, five-year dataset runs into tens of millions of rows. Below the price itself there is a second, even faster layer of information the order book which can change completely within seconds.

People who study or trade these markets care about much more than the current price. They want to know how volatile an asset has been, how likely a big drop is, whether a pattern like “the price just fell 5% in an hour” is usually followed by a recovery or by a further fall, and how tightly assets move together in calm versus stressed periods. Answering these questions needs views of time, statistics, and market structure together, on the same data.

The problem is that the raw data is very hard to use directly. Exchange APIs return dense JSON; the historical archives come as thousands of compressed CSV files; and the live order book is a stream of updates arriving ten times per second. Having the data is not enough it must be made explorable and understandable. That is exactly what visual analytics does: it turns raw candles and order-book snapshots into linked, interactive charts that people can reason with.

Our motivation for CryptoLens is to build one self-contained platform where a user can start from a five-year overview, brush down to a single hour, immediately see the risk numbers and order-book shape for that window, and test a trading idea against the full history all without writing a line of code.

2.2 Problem Statement

High-quality crypto market data is freely available today, from sources like the Binance Vision portal and public WebSocket streams. But in raw form it is out of reach for most people: millions

of rows, compressed archives, and streaming feeds need real programming and data-engineering skill to use. As a result, the insight hidden in this free data mostly goes unused.

The core problem this project addresses is the lack of a **single, interactive, linked platform** for exploring both the *time* dimension and the *structure* dimension of crypto market data. Charting websites exist, but they usually show only the price chart: no risk analytics, no pattern back-testing, and each asset viewed on its own.

Specific gaps identified:

- **No multi-resolution exploration:** most tools fix one timeframe. There is rarely a smooth way to move from a five-year overview down to minute candles while every other panel stays in sync.
- **No built-in risk analytics:** charting tools show price but not risk. Users cannot see Value-at-Risk or how far real returns deviate from a normal (bell-curve) model in stressed markets.
- **No pattern back-testing:** there is no easy, interactive way to ask “after this kind of move, what happened next?” and see the full range of historical outcomes.
- **No order-book view next to history:** live depth and historical price are almost never shown together, so the link between standing liquidity and price is hard to study.
- **No cross-asset view:** assets are shown one at a time, with no correlation matrix or network to reveal how they move together.

Because of these gaps, freely available crypto market data is underused for research, education, and informed decision-making.

2.3 Scope and Objectives

The project covers minute-level market data for five major crypto assets over five years (2020–2024). The emphasis is on interactivity, linking between views, and analytical depth.

Project objectives:

- Load and store roughly 10.5 million one-minute candles from the public Binance Vision portal into a fast, embedded columnar database.
- Build a **multi-resolution temporal explorer** with semantic zoom, so any span from five years down to one day draws as a readable candle chart.
- Build a **risk analytics engine** that computes volatility, Value-at-Risk, and Conditional VaR under normal, stress, and crash scenarios.
- Build a **pattern tester** that scans all history for a user-defined trigger and shows the spread of what happened afterwards.
- Show **order-book depth**, both live (through a WebSocket relay) and rebuilt for historical timestamps.
- Show **cross-asset correlation** as a heatmap and as a node-link network, with a rolling-correlation drill-down for any pair.
- Link every view through one shared, brushed time window (“brush-and-link”).

Target users:

- **Quantitative researchers and analysts:** to study volatility regimes, tail risk, correlation structure, and how price patterns have behaved historically.

- **Traders:** to judge risk before taking a position, check order-book liquidity, and test whether a trading setup actually had an edge in the past.
- **Educators and students:** to learn about market microstructure and risk measures by exploring real data hands-on.
- **The general public:** anyone curious about how crypto markets have behaved over time.

2.4 Theoretical Basis: Measuring Risk and Co-Movement

The basic chart-reading terms (candles, order book, walls) were covered in the Introduction. Here we explain the statistical ideas the tool computes with.

Log returns. CryptoLens works with *log returns*, $r_t = \ln(P_t/P_{t-1})$, a standard variant of the percentage return. Log returns can be added across time and are better behaved statistically. The distribution of these returns its average, its spread (volatility), and the shape of its tails describes an asset's risk.

Annualized volatility. Volatility is the standard deviation of returns. To compare volatilities measured at different resolutions, they are *annualized*: multiplied by $\sqrt{N}$, where N is the number of intervals in a year (for crypto: 525,600 minutes, 8,760 hours, or 365 days the market never closes).

Value-at-Risk (VaR) and Conditional VaR (CVaR). **VaR 95%** answers: “how big is the loss that is only exceeded 5% of the time?” **VaR 99%** asks the same for the worst 1%. **CVaR** (also called Expected Shortfall) goes one step further: it is the *average* loss inside that worst tail so it captures how bad the bad days are, not just how often they happen. Real crypto returns have much fatter tails than a normal (bell-curve) model predicts, so CryptoLens estimates these numbers under three different regimes (normal, stress, crash) using Monte Carlo simulation.

Rolling correlation. Correlation between two assets is not fixed it rises and falls over time, and typically jumps toward +1 during crashes. CryptoLens therefore shows both a single correlation number for the selected window and a *rolling* correlation line that tracks how the relationship changed inside that window.

2.5 Analyzing Crypto Market Data

Crypto market datasets typically contain, per candle:

- **Timestamp** of the interval (open time)
- **Open, High, Low, Close** prices
- **Volume** (quantity traded)
- Sometimes **quote volume** and **trade count**

By studying this data over time, one can analyze:

- Trends and regimes in price and volatility
- The shape of the return distribution, especially its tails
- Tail risk (VaR/CVaR) and the probability of large drops
- Correlation structure across assets
- Order-book liquidity and its relationship to price

Challenges in crypto market data analysis. The data is plentiful, but it is not easy to work with:

- a. **Data volume:** five years of minute data across five assets is about 10.5 million rows. Naive tools freeze; the system needs a columnar store and on-the-fly aggregation to stay fast.
- b. **Inconsistent archive formats:** Binance's monthly files are not uniform older files have no header row, and some 2025-era files changed the timestamp unit from milliseconds to microseconds. The pipeline must detect and fix both automatically.
- c. **Fat tails:** crypto returns are far more extreme than a normal distribution predicts. Risk numbers that assume normality badly understate the chance of large moves this is why the stress and crash scenarios exist.
- d. **Fragile live feed:** the live order book depends on a persistent WebSocket connection to Binance, which can drop, restart slowly, or be geo-blocked (Binance rejects US-region connections with HTTP 451). The system must fail gracefully.

3 Tasks

This section lists the specific goals the dashboard addresses:

- Smooth multi-resolution exploration of price history.
- Measuring and visualizing market risk, including tail risk under several regimes.
- Back-testing price patterns across the full historical record.
- Showing order-book structure, both live and historical.
- Revealing cross-asset relationships through correlation analysis.
- Linking all of the above through one shared, brushed time window.

3.1 Tasks Addressed by the Visualization Tool

- Multi-resolution candlestick + volume charts with semantic zoom and a range-slider brush.
- Monte-Carlo risk analytics (VaR, CVaR, drop probabilities) with normal, stress, and crash scenarios.
- Interactive pattern back-testing with a fan chart of forward returns.
- Live and reconstructed order-book depth charts.
- A cross-asset correlation heatmap and node-link network with rolling drill-down.
- Brush-and-link coordination, so a selection in one view updates all others.

4 Proposed Methodology of Dedicated Dashboard Views and Results

Visual analytics bridges the gap between raw market data and real understanding. An effective dashboard turns millions of candles and a live order book into **clear, interactive, linked visuals** that support trading, research, and education.

This project uses a modern web stack **FastAPI** + **DuckDB** on the backend, a **Node.js WebSocket relay** for live data, and **React** + **Plotly/D3** on the frontend. Together these enable:

- Real-time filtering and brushing across a shared time window

- On-the-fly candle aggregation at five resolutions
- A responsive, single-screen linked layout

4.1 Purpose of Visualization in Market Analysis

The dashboard is designed with several goals:

- **Pattern discovery:** spot trends, volatility clusters, and repeating price behaviour.
- **Risk identification:** surface tail risk, fat-tailed return distributions, and the chance of large drops.
- **Hypothesis testing:** let users ask “what happens after X?” and see the historical answer as a full range of outcomes, not a single number.
- **Decision support:** give traders and researchers the statistical and structural context needed to reason about a position.

4.2 Dedicated Dashboard Views and Their Functionalities

The dashboard provides **five dedicated views**, each with a focused purpose.

1. The Time Machine (temporal explorer)

- A compressed five-year overview line chart with a range-slider brush.
- A detail candlestick + volume chart whose resolution is chosen automatically or forced manually.
- Semantic zoom: the resolution adapts to the span (5Y → monthly, 1Y → weekly, 1M → daily, 1W/1D → hourly, below a day → minute).

2. Market Depth Visualiser

- A mirrored depth chart: buy-side support against sell-side resistance.
- Live mode streams the real Binance order book through a WebSocket relay.
- Historical mode rebuilds a plausible depth shape from price and volume.

3. The What-If Simulator

- A pattern tester: define a trigger (a drop or spike of $X\%$ over N periods) and a forward horizon.
- Scans all history (or just the brushed window) for every occurrence.
- Shows the spread of what happened next as a percentile fan chart.

4. Volatility Engine

- Rolling annualized volatility and the full return distribution.
- VaR 95/99 and CVaR 95 under normal, stress, and crash scenarios, via Monte Carlo simulation.
- Drop-probability bars and an observed-versus-normal-model histogram.

5. Cross-Asset Correlation Matrix

- A 5×5 pairwise correlation heatmap over log returns.
- A node-link network view where edge thickness encodes correlation strength.
- Click any pair to drill into its rolling correlation over the window.

4.3 Tools and Technologies

- **FastAPI:** a fast Python web framework serving the REST API.
- **DuckDB:** an embedded, columnar analytical database that handles 10.5M rows with zero server setup.
- **pandas, NumPy, SciPy:** for aggregation, returns, and the statistics engine (Monte Carlo, distributions, correlations).
- **Node.js + ws:** a relay that keeps live Binance depth streams open and shares them with all browser clients.
- **React + Vite:** the single-page frontend, with global state driving the shared window.
- **Plotly.js and D3/SVG:** for interactive candlesticks, heatmaps, fan charts, and the correlation network.

4.4 Outcome

This modular, linked design lets users explore crypto market data at **temporal**, **statistical**, and **structural** scales on a single screen making it both a teaching tool and a practical asset for research and trading.

5 Data Sources and Preprocessing

5.1 Overview of Datasets

The project uses two data sources: a large historical archive for all temporal and statistical views, and a live streaming feed for the real-time order book.

5.1.1 Binance Vision: Historical 1-Minute Klines

The primary dataset comes from the public **Binance Vision** data portal (data.binance.vision), which publishes monthly archives of one-minute candlesticks for every trading pair free, with no API key or login required.

Key features of the dataset:

- **Time period:** January 2020 to December 2024 (SOL starts at its August 2020 listing).
- **Assets:** BTC, ETH, BNB, SOL, XRP (traded against USDT).
- **Resolution:** one-minute OHLCV candles.
- **Format:** monthly ZIP-compressed CSV files.
- **Volume:** about 10.5 million rows for the planned range (verified locally at 12.8M rows across the five assets).

Important columns (kept from the raw Binance kline schema):

Column	Description
symbol	Display symbol of the asset (e.g. BTC, ETH)
ts	Candle open time, normalised to a UTC-naive timestamp

Column	Description
open, high, low, close	OHLC prices for the one-minute interval
volume	Quantity traded in the interval
quote_volume	Value traded in the interval, in USDT
trades	Number of individual trades in the interval

Its size, uniform resolution, and completeness make this dataset well suited for temporal exploration, risk analysis, pattern back-testing, and correlation analysis.

5.1.2 Binance Live Depth Stream

For the live order book, the **Binance public depth WebSocket** (`wss://stream.binance.com:9443/ws/<pair>`) supplies the top 20 bid and ask levels every 100 milliseconds, again with no login.

Purpose in the project:

- Powers the real-time “live” mode of the Market Depth Visualiser.
- Provides genuine order-book snapshots for structural analysis.
- Serves as the reference against which the historical depth reconstruction was designed.

5.2 Data Cleaning and Aggregation

The raw Binance archives needed several cleaning steps. These are done by the ingestion pipeline (`ingestion/download.py`) and the DuckDB access layer (`app/store.py`).

5.2.1 Heterogeneous Header Detection

Older monthly files have no header row, while newer ones do. The pipeline inspects the first bytes of each file and applies the raw Binance column names only when no header is present:

```
has_header = raw[:9].lower().startswith(b"open_time")
return pd.read_csv(io.BytesIO(raw),
    header=0 if has_header else None,
    names=None if has_header else _RAW_COLS)
```

5.2.2 Timestamp Unit Normalisation

Historical files store the open time in milliseconds, but some 2025-era files switched to microseconds. The unit is detected from the size of the first value and converted to a UTC-naive datetime:

```
sample = float(series.iloc[0])
unit = "us" if sample > 1e14 else "ms"
return pd.to_datetime(series.astype("int64"), unit=unit)
```

5.2.3 Deduplication on Insert

Each candle is uniquely identified by the pair (`symbol`, `ts`), enforced as a primary key. Because ingestion can be re-run or months can overlap, rows are inserted only if they are not already present so running the pipeline twice never creates duplicates:

```
INSERT INTO klines_1m
SELECT * FROM _incoming
WHERE (symbol, ts) NOT IN (SELECT symbol, ts FROM klines_1m)
```

5.2.4 On-the-Fly Multi-Resolution Aggregation

Instead of pre-computing every resolution, coarser candles are built at query time from the one-minute base table using DuckDB's `time_bucket`. Open takes the first value in the bucket, high the maximum, low the minimum, close the last value, and volume the sum:

```
SELECT time_bucket(INTERVAL '1 hour', ts) AS ts,
       arg_min(open, ts) AS open,
       max(high) AS high,
       min(low) AS low,
       arg_max(close, ts) AS close,
       sum(volume) AS volume
FROM klines_1m
WHERE symbol = ? AND ts BETWEEN ? AND ?
GROUP BY 1 ORDER BY 1
```

This keeps storage minimal (one base table) while letting any of the five resolutions (1m, 1h, 1d, 1w, 1M) be served instantly.

5.3 Feature Engineering

To prepare the raw candles for statistical and visual analysis, the statistics engine (`app/stats.py`) computes the following derived quantities on demand.

5.3.1 Log Returns

The foundation of all risk and correlation analysis is the log return series, computed from close prices and used everywhere downstream:

```
def log_returns(close: pd.Series) -> pd.Series:
    return np.log(close / close.shift(1)).dropna()
```

5.3.2 Rolling Volatility

Volatility is computed as a rolling standard deviation of returns and annualized by $\sqrt{N}$, where N is the number of periods per year for the chosen resolution ($N = 525,600$ for minutes, 8,760 for hours, 365 for days).

5.3.3 Distribution Moments

For each window the engine computes the mean, standard deviation, **skew** (whether the distribution leans toward gains or losses), and excess **kurtosis** (how heavy its tails are), via SciPy. High kurtosis is the number behind the “fat tails” that make crypto risky, and it drives the observed-versus-normal comparison in the Volatility Engine.

5.3.4 Scenario Sampling

To estimate tail risk beyond what history alone shows, the engine draws 200,000 Monte Carlo samples per scenario. The **normal** scenario uses a Gaussian (bell-curve) model at the observed mean and volatility; **stress** uses a fatter-tailed Student- t ($df = 5$) at about $1.6\times$ the observed

volatility; and **crash** uses a very heavy-tailed Student- t ($df = 3$) at about $2.5\times$ volatility with a downward drift.

5.4 Specific Dataset Notes

The following dataset-specific issues came up during ingestion and analysis.

5.4.1 Missing Months and Late Listings

Not every asset existed for the whole range SOL, for example, only lists from August 2020. Months that do not exist upstream return HTTP 404 and are simply skipped instead of stopping the ingestion:

```
resp = session.get(url, timeout=60)
if resp.status_code == 404:
    return None # month not available yet -> skip
```

5.4.2 Response Size Guards

A hand-crafted API request could otherwise ask for hundreds of thousands of candles and freeze the browser. The backend therefore caps any OHLCV response at 3,000 candles and tells the caller to pick a coarser resolution. The frontend independently disables any resolution that would draw fewer than 6 or more than 2,000 candles for the current window.

5.4.3 Synthetic vs. Real Depth

A live order book only exists for the present moment. For any historical timestamp there is no real book, so the Market Depth view rebuilds a plausible depth shape (see Section 8.2) and clearly labels it *synthetic (reconstructed) not real quotes*, so users never mistake it for genuine historical liquidity.

6 Application Architecture

6.1 Framework: A Three-Tier Web Application

CryptoLens is built as three cooperating services, each chosen for what it does best:

- **Backend FastAPI + DuckDB (Python):** a REST API that reads the DuckDB database and runs the statistics engine. FastAPI provides clean, validated endpoints; DuckDB stores millions of rows with no database server to set up.
- **Live relay Node.js + ws:** a lightweight relay that keeps one upstream Binance depth stream open per symbol and shares normalised snapshots with all connected browsers.
- **Frontend React + Vite + Plotly/D3:** a single-page application that draws all five views and links them through shared state.

Key reasons for this split:

- **Right tool per tier:** heavy analytical queries run in DuckDB; long-lived sockets live in Node; interactive drawing lives in the browser.
- **Zero-setup storage:** DuckDB is embedded, so the whole stack runs without a database server, and swapping to a server database later is mechanical.
- **Independent deployment:** each tier can be deployed and scaled on its own.

6.2 Core Components: Layout, State, and Callbacks

Backend endpoints. The API (`app/main.py`) exposes a small, focused set of endpoints under `/api`:

Endpoint	Purpose
GET <code>/health</code>	Liveness check
GET <code>/meta</code>	Coverage, symbols, and a suggested default window
GET <code>/ohlcv</code>	Candlesticks at a resolution (auto or forced)
GET <code>/volatility</code>	Risk report + normal/stress/crash scenarios
GET <code>/correlation</code>	5×5 correlation matrix for a window
GET <code>/correlation/pair</code>	Rolling correlation for one pair
GET <code>/patterns</code>	What-if pattern back-test
GET <code>/depth</code>	Reconstructed historical order book

Historical responses never change, so they are cached (LRU cache) for the life of the process.

Frontend state and “callbacks.” The app is organised into five panels, linked by one global state context (`state.jsx`) instead of per-tab callbacks. The shared state holds the active **asset**, the brushed **window**, the selected correlation **pair**, the correlation view mode, and the What-If trigger **events**. A small `useFetch` hook re-runs the relevant API call whenever its inputs (asset, window, form values) change listening to inputs, fetching filtered data, and updating the charts without a page refresh, much like Dash callbacks do.

Brush-and-link. The Time Machine’s range-slider writes the shared **window**; every other view reads it and refetches. In the other direction, the What-If Simulator writes its trigger timestamps back into shared state, and the Time Machine draws them as markers on its candlestick chart a two-way linked-view interaction.

6.3 UI and Styling Elements

The interface uses a clean **light theme** tuned for WCAG-AA contrast, with the industry-standard green/red palette for up/down candles. Other UI features:

- **Single-screen linked layout:** all five panels fit one screen and animate in with a staggered entrance.
- **User controls:** preset zoom buttons (5Y/1Y/1M/1W/1D), resolution toggles, scenario selectors (normal/stress/crash), view-mode switches (matrix/network, historical/live), and a pattern-definition form.
- **State rings and live badges:** panels glow to show state (for example a connected live feed, or a crash scenario selected), and the depth panel shows a pulsing live indicator.
- **Informative tooltips:** hovering shows exact values (prices, correlations, probabilities) without cluttering the screen.

7 Overall Application Story: Understanding Crypto Market Behavior

CryptoLens is designed as a guided journey through the behaviour of a crypto asset. It combines the time, risk, and structure layers of analysis into one connected experience.

The **Time Machine** shows how price and volume have evolved over five years. A range-slider brush over a compressed overview drives a detail candlestick chart whose resolution adapts automatically to the selected span from a multi-year trend down to a single volatile hour.

From there, the **Volatility Engine** turns to risk. For exactly the window the user has brushed, it computes rolling volatility, the shape of the return distribution, and tail-risk numbers under normal, stress, and crash regimes turning an abstract price move into concrete Value-at-Risk figures.

The **What-If Simulator** lets users question history directly: define a trigger such as “a 5% drop in one hour,” and the tool finds every past occurrence and shows the range of what happened next. Those trigger points are also drawn back onto the Time Machine’s chart, tying the statistics to specific moments on the timeline.

The **Market Depth Visualiser** adds the structural layer the standing buy and sell orders around the current price live from Binance, or rebuilt for a historical window. Finally, the **Correlation Matrix** widens the lens from one asset to all five, showing how they move together and how that changes over time.

Underneath it all is a robust data pipeline: millions of one-minute candles cleaned and deduplicated into a columnar store, aggregated on the fly, and enriched with log returns, distribution statistics, and Monte-Carlo scenarios. Together these modules tell one coherent story from years of price history down to the order book at a single moment supporting exploration, risk assessment, and education.

8 View-Specific Analysis and Features

8.1 The Time Machine (Temporal Explorer)

The Time Machine enables temporal analysis of price across different time scales. It has two linked parts: a compressed **overview** (a five-year close-price line with a range-slider) and a **detail** candlestick + volume chart. Together they show how price and trading activity evolve over time.

8.1.1 Interaction and Controls

The interface provides several interactive controls:

- **Range-slider brush:** dragging the handles on the overview sets the shared time window that drives the whole dashboard. Updates are debounced (250 ms) so the four linked panels do not refetch on every tiny movement of the drag.
- **Quick-zoom presets:** 5Y, 1Y, 1M, 1W, and 1D buttons jump the window to the last N days of available data.
- **Resolution override:** an **Auto** mode plus manual 1m/1h/1d/1w/1M buttons. A button is enabled only when the current window would give a readable number of candles (between 6 and 2,000); otherwise it falls back to Auto.

8.1.2 Semantic Zoom (Auto Resolution)

In Auto mode the backend maps the selected span to a resolution that keeps the chart readable roughly 30–90 candles at any zoom level:

Span	Resolution	Approx. candles
more than 1.1 years	monthly	5Y $\approx$ 60
2.5 months–1.1 years	weekly	1Y $\approx$ 52

Span	Resolution	Approx. candles
8–75 days	daily	1M $\approx$ 30
1–8 days	hourly	1D $\approx$ 24
less than 1 day	minute	

8.1.3 Candlestick and Volume Visualization

The detail chart draws standard OHLC candlesticks (green up, red down) with a volume panel beneath, coloured to match each candle's direction. What-If trigger timestamps that fall inside the current window are drawn as markers above the candles, linking the pattern scan back to the timeline.

8.1.4 Backend and Plot Logic

Both charts request data from the `/api/ohlcv` endpoint, which aggregates candles on the fly from the one-minute base table using `time_bucket`. Responses are cached on both sides (an LRU cache on the server, a FIFO cache in the browser), so switching presets or resolutions back and forth redraws instantly instead of re-hitting the backend. Timestamps are kept timezone-naive from end to end; without this, UTC conversions made the brush drift away from the pointer during a drag.

8.1.5 Insights and Use Cases

- **Regime discovery:** spot bull and bear phases and volatility clusters across years.
- **Event zoom:** brush to a specific crash or rally and inspect it minute by minute.
- **Linked analysis:** every brush instantly refocuses the risk, correlation, and depth panels on the same window.

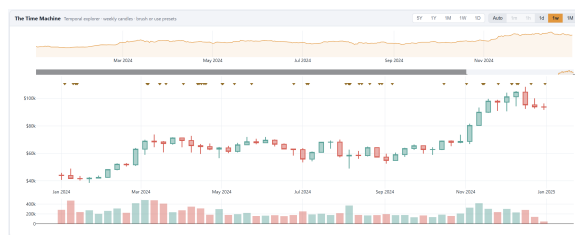


Figure 3: The Time Machine five-year overview with range-slider brush and detail candlestick + volume chart.

To show the semantic zoom in action, Figure 4 presents the same asset at four different resolutions.



(a) Monthly (1M) candles 5Y span



(b) Weekly (1w) candles 1Y span



(c) Daily (1d) candles 1M span



(d) Minute (1m) candles sub-day span

Figure 4: Semantic zoom: the same price history rendered at four resolutions. As the brushed span shrinks, the Time Machine automatically switches from monthly to weekly, daily, and finally minute candles.

8.2 Market Depth Visualiser

The Market Depth Visualiser shows the order book as a **mirrored depth chart**: buyer volume (support) piles up on the bid side and seller volume (resistance) on the ask side, meeting at the mid price. Its purpose is to expose the liquidity standing around the current price.

8.2.1 Functionality Overview

- **Historical mode:** for the end of the brushed window, the backend rebuilds a plausible depth shape from recent price and volume.
- **Live mode:** the panel subscribes to the WebSocket relay and streams the real Binance top-20 order book, updating continuously.

8.2.2 Live Streaming and Resilience

Live mode connects to the Node relay, which keeps one upstream Binance depth stream open per symbol (@depth20@100ms) and shares normalised, cumulative snapshots with all subscribed clients. The client **auto-reconnects with exponential backoff** (1, 2, 4, 8, 15 s), so a dropped socket, a Wi-Fi blip, or a slow free-tier restart recovers by itself instead of hanging. If Binance geo-blocks the upstream (HTTP 451 from US regions), the relay sends a status message and the UI shows a clear “live feed unavailable” state instead of an endless spinner.

8.2.3 Historical Reconstruction

For past timestamps there is no real order book, so a depth shape is generated around the close price, scaled by recent traded volume, with level sizes that shrink smoothly away from the mid price and a couple of randomly placed “walls.” The spacing between levels is about 0.05% of the price:

```
tick = max(price * 0.0005, 1e-9) # ~5 bps spacing
base = max(recent_volume, 1.0) / levels
decay = np.exp(-i / levels) # size decays away from mid
```

```
if i == wall_bid: size_b *= 6.0 # occasional liquidity wall
```

This is clearly labelled *synthetic* in the UI an approximation for building intuition about structure, not real quotes.

8.2.4 Interactive Elements

- **Mode toggle:** switch between the historical reconstruction and the live feed.
- **Mid-price marker:** a dotted vertical line marks the point between the best bid and the best ask.
- **Live badge:** a pulsing indicator with the connection state (connecting / live / reconnecting / unavailable) and a live-updating mid price.

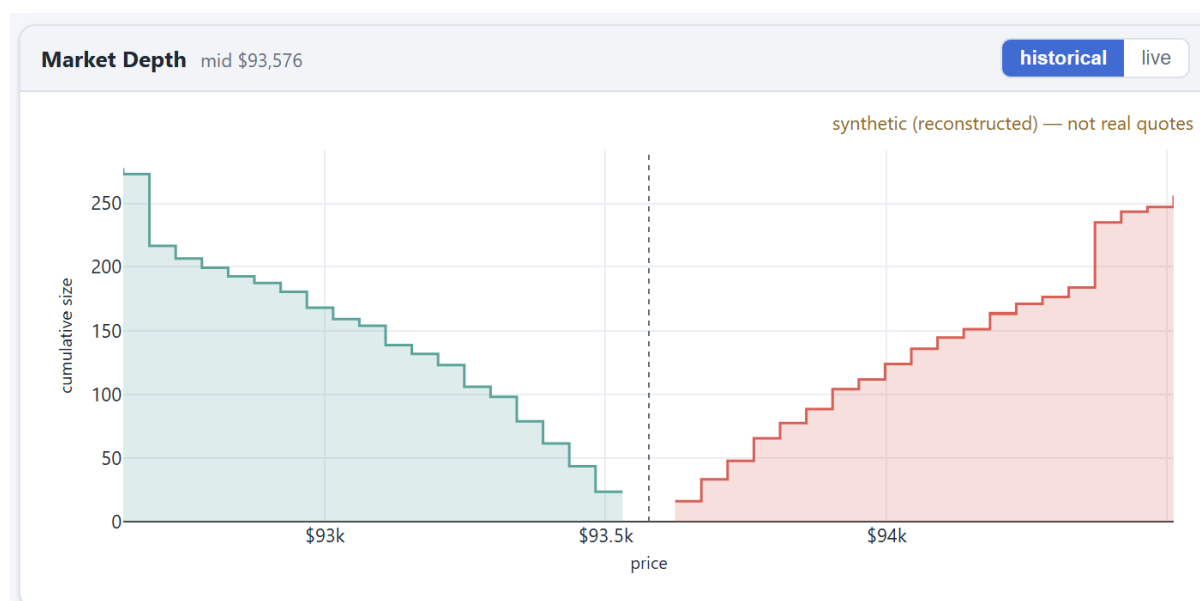


Figure 5: Market Depth Visualiser mirrored bid/ask depth around the mid price.

8.3 The What-If Simulator

The What-If Simulator is a **pattern tester**. The user defines a price trigger and a forward horizon; the tool then scans the historical record for every occurrence of that trigger and shows the spread of what happened afterwards.

8.3.1 Functionality Overview

The user specifies: a **direction** (price drops or spikes), a **threshold** (e.g. at least 2%), a **lookback** (over how many periods the move is measured), a **horizon** (how many periods forward to follow), a **resolution** (1h/1d/1m), and a **scope** (all history or just the brushed window). Pressing *Run scan* evaluates the pattern against the data.

8.3.2 Core Visualization: The Fan Chart

Results are shown as a **percentile fan chart** of forward returns after each trigger: a wide band covering the 10th–90th percentile of outcomes, a narrower 25th–75th band inside it, a solid median line, and a dotted mean line. The fan opens up like a triangle as the horizon grows

showing that the further ahead you look, the wider the range of possible outcomes becomes. This makes the *range* of history visible, not just an average.

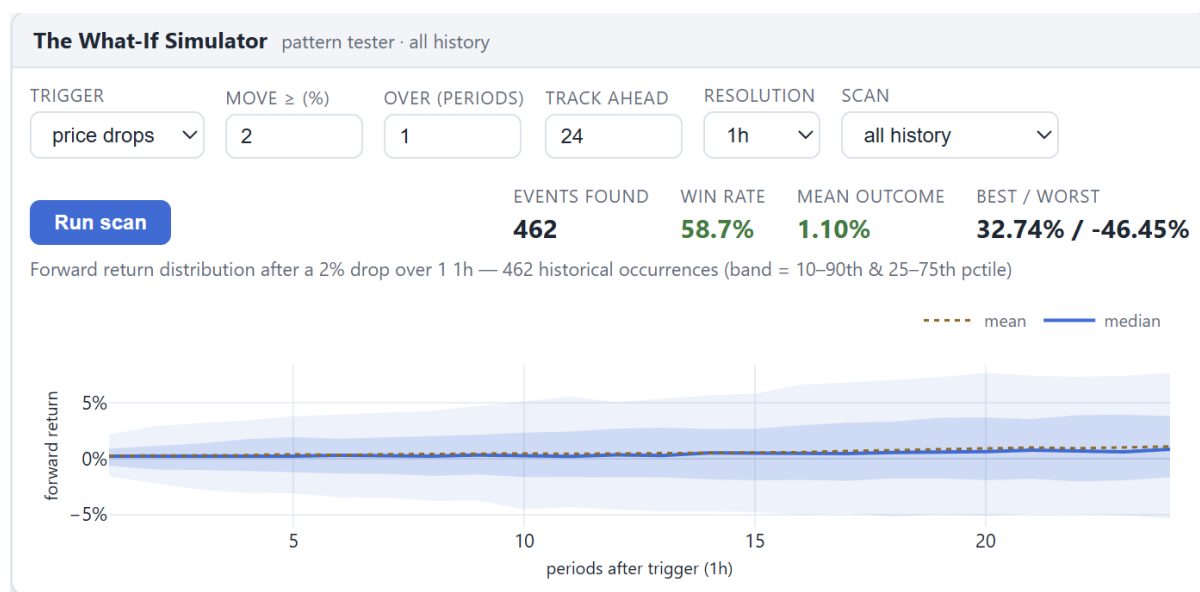


Figure 6: The What-If Simulator pattern form, scan statistics, and forward-return fan chart.

8.3.3 Scan Algorithm and Statistics

For every occurrence of the trigger, the engine follows the forward return at each step of the horizon and records the final outcome. Next to the chart it reports the number of events found, the **win rate** (the fraction of positive final outcomes), the mean outcome, and the best and worst cases:

```
base_ret = values[lookback:] / values[:-lookback] - 1.0
trigger = base_ret <= -threshold # 'drop'
paths[:, k-1] = values[pos + k] / values[pos] - 1.0 # forward return
pct = np.percentile(paths, [10, 25, 50, 75, 90], axis=0)
```

8.3.4 Interactive Elements and Linking

- **Pattern form:** direction, threshold, lookback, horizon, resolution, and scope controls.
- **Live statistics:** events found, win rate, mean outcome, best/worst, animated on each scan.
- **Brush-and-link:** trigger timestamps are written to shared state and drawn as markers on the Time Machine's chart, connecting the statistics to specific points in the price history.

8.4 Volatility Engine

The Volatility Engine provides market-risk analytics for the brushed window. It measures how volatile an asset has been and how heavy its tail risk is under different market regimes.

8.4.1 Functionality Overview

The engine reports **annualized volatility**, a **rolling volatility** line, and tail-risk numbers **VaR 95%**, **VaR 99%**, and **CVaR 95%** each estimated under three scenarios. It also describes the shape of the return distribution through its skew and excess kurtosis.

8.4.2 Scenario Modelling

Real crypto returns have far fatter tails than a normal (bell-curve) model, so risk is estimated by Monte Carlo simulation (200,000 samples) under three regimes:

- **Normal:** a Gaussian model at the observed mean and volatility.
- **Stress:** a fatter-tailed Student- t ($df = 5$) at about $1.6\times$ the observed volatility.
- **Crash:** a very heavy-tailed Student- t ($df = 3$) at about $2.5\times$ volatility with a downward drift.

The three scenarios are colour-coded like a traffic light (green / amber / red), tuned for contrast and colour-blind readability, and the whole panel glows to match the active scenario.

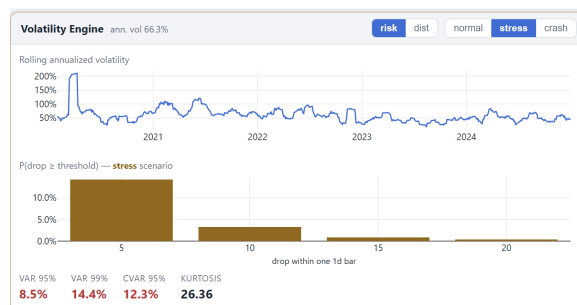
8.4.3 Core Visualizations

- **Rolling volatility:** an annualized-volatility line over the window.
- **Drop-probability bars:** the modelled chance of a single-bar drop of 5%, 10%, 15%, and 20% under the selected scenario.
- **Return distribution (dist tab):** a histogram of observed returns with the normal-model curve drawn on top at the same mean and spread the visible gap in the tails is exactly what the reported kurtosis measures.

Figure 7 shows the panel's tabs and scenarios side by side.



(a) Risk tab normal scenario



(b) Risk tab stress scenario



(c) Risk tab crash scenario



(d) Dist tab observed vs. normal model

Figure 7: Volatility Engine tabs. The risk tab under the three scenarios (note how VaR/CVaR and the drop probabilities grow from normal to crash), and the dist tab comparing observed returns with the normal model.

8.4.4 Insights and Use Cases

- **Regime-aware risk:** compare how VaR and CVaR balloon from the normal to the crash scenario.
- **Fat-tail intuition:** see directly how far real returns exceed the normal model.
- **Position sizing:** use tail-risk numbers computed for the exact window under study.

8.5 Cross-Asset Correlation Matrix

The Correlation Matrix shows how the five assets move together over the brushed window. Unlike the other views, it always shows all five assets, whichever one is active.

8.5.1 Core Visualizations

- **Heatmap:** a 5×5 pairwise correlation matrix over log returns, drawn on a diverging red–blue scale (-1 to $+1$) with the value printed in each cell.
- **Network view:** the same matrix as a node-link diagram. Each asset is a node; each edge's thickness and opacity encode the correlation strength $|\rho|$, and its colour encodes the sign (blue positive, red negative). The layout is deliberately 2D the data has no third dimension, so adding perspective would only distort it.

8.5.2 Interactive Elements and Drill-Down

- **Click to drill down:** clicking a heatmap cell (or a network edge) selects that pair and reveals its **rolling correlation** over the window, plus the overall correlation number.
- **Click a node:** in network mode, clicking a node makes that asset the active one across the whole dashboard.
- **View toggle:** switch between the matrix and network views.

Figure 8 shows the three ways of reading the same correlation data.

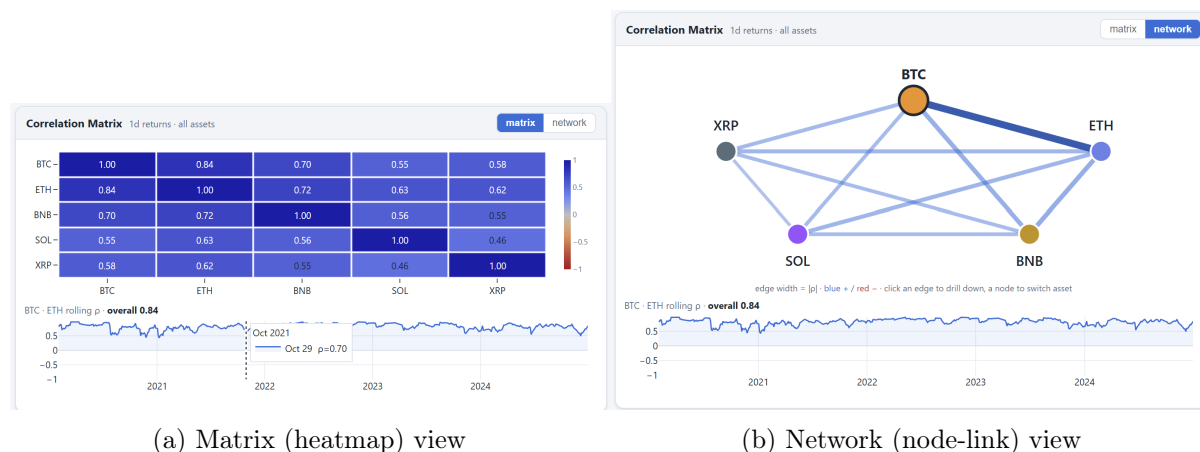


Figure 8: Cross-Asset Correlation Matrix: the heatmap view, the network view, and the rolling-correlation drill-down for one selected pair.

8.5.3 Backend Logic

The matrix is computed from a wide table of bucketed close prices via pairwise log-return correlation (`/api/correlation`); the drill-down uses a 30-period rolling correlation for the selected pair (`/api/correlation/pair`). Both adapt their candle resolution to the window using the same semantic-zoom rule as the Time Machine.

8.5.4 Insights and Use Cases

- **Diversification:** spot which assets are least correlated in the window.
- **Regime shifts:** watch correlations rise toward 1 during stress (assets “move together” in crashes) via the rolling drill-down.
- **Structure at a glance:** the network view makes the strongest and weakest relationships immediately visible.

8.6 Member Contributions

Team Member (Roll No.)	Contribution
Soumya Jhunjhunwala (241034)	Built the data-ingestion script that downloads five years of minute-level candles for all five coins directly from Binance’s public archive, and loads them into the DuckDB database the whole app reads from.
Vivek Singh (241186)	Built the backend’s OHLCV endpoint, including the logic that automatically picks a sensible candle size (minute/hour/day/week/-month) based on how wide a time window is selected, so charts stay readable at any zoom level.
Pratyush Sandhwar (240788)	Built the Node.js WebSocket relay that connects to Binance’s live order-book stream, converts the raw bid/ask levels into cumulative depth, and pushes updates out to every connected browser in real time.
Aman (240097)	Set up the overall React application: the page layout, the shared state that keeps the selected asset, time window, and drill-down pair in sync across all five views, and the API client that connects the frontend to the backend.
Arpit Kumar Singh(240194)	Built the Time Machine view: the draggable mini-chart at the top for picking a time range, and the detailed candlestick and volume chart below it that automatically switches resolution as the selected range changes.
Lakshya Arukiya (240589)	Built the Market Depth view: the mirrored buy/sell depth chart, the live mode that plots real-time Binance data, and the historical mode that reconstructs an approximate order book for past dates.

Team Member (Roll No.)	Contribution
Aadi Sehal (240005)	Built the What-If Simulator: lets a user define a price-movement rule (e.g. “price drops 5% in an hour”), scans the full five-year history for every matching instance, and shows what typically happened afterwards, with a win-rate and best/worst outcomes.
Anandan Iyer (240117)	Built the Volatility Engine and Correlation Matrix: rolling volatility and risk charts under normal/stress/crash scenarios, plus the five-asset correlation heatmap and an alternate node-link view, with click-to-drill-down on any pair.
Tulika (241106)	Handled integration and testing across all five views, managed the GitHub repository, and set up the multi-service deployment (backend, WebSocket relay, and frontend) for the live demo.

9 Conclusion

CryptoLens is a comprehensive, interactive platform for exploring cryptocurrency market data. By turning millions of raw one-minute candles and a live order-book stream into linked visual stories, it lets users uncover patterns in time, statistics, and structure with ease from multi-year price regimes to the order book at a single moment, from tail-risk estimates to the historical behaviour of price patterns.

The three-tier architecture cleanly separates data ingestion, analytical querying, live streaming, and interactive drawing. This keeps the application maintainable and easy to extend, while each of the five views offers its own perspective:

- The **Time Machine** makes five years of history explorable at any resolution.
- The **Volatility Engine** measures risk under normal, stress, and crash regimes.
- The **What-If Simulator** turns trading ideas into distributions of historical outcomes.
- The **Market Depth Visualiser** exposes order-book structure, live and historical.
- The **Correlation Matrix** shows how assets move together and how that changes over time.

Special attention went into interactivity and linking. The brush-and-link design one shared time window driving every panel, with the pattern scanner writing markers back onto the timeline makes the dashboard one connected workspace instead of a set of separate charts. CryptoLens shows the value of visual analytics in making complex, high-frequency financial data understandable, and lays a foundation for future work in real-time risk monitoring and prediction.

10 Code and Deployment Links

- **GitHub repository:** <https://github.com/lakshya-ar/CryptoLens-Financial-Visual-Analytics>
- **Deployment:** the stack is deployed as three services a FastAPI + DuckDB API, a Node WebSocket relay, and a static React frontend via a Render Blueprint (`render.yaml`), hosted in the Singapore region so the live Binance feed is not geo-blocked. The frontend can also

be deployed to Vercel.

Live application: <https://cryptolens-frontend-u6jo.onrender.com>

End of report